

Билет 8. Мьютекс на трёх состояниях.

Мотивация: меньше сисколлов и проблема

OneShot

0. Формулировка билета

Мьютекс на трёх состояниях. Мотивировка: уменьшение числа сисколлов, проезд по памяти в деструкторе примитива OneShot. Реализация мьютекса на трёх состояниях.

Главная идея билета:

Двухсостояниевый futex-мьютекс показывает базовую идею, но у него есть проблемы.

Он всегда вызывает `futex_wake` в `unlock`, даже если никто не ждёт. Кроме того, между `store(0)` и `futex_wake` мьютекс уже считается свободным, и объект с этим мьютексом может быть уничтожен.

Мьютекс на трёх состояниях хранит в одном атомарном слове не только “свободен/занят”, но и информацию “есть ожидающие”. Это позволяет не вызывать `futex_wake`, когда ожидающих нет, и уменьшает проблему обращения к памяти после освобождения.

Ключевая идея:

0 – свободен
1 – занят, ждущих нет
2 – занят, есть или могли быть ждущие

1. Вспоминаем двухсостояниевый futex-мьютекс

В прошлом билете была простая реализация:

```
struct Mutex {
    std::atomic<int> state{0}; // 0 – free, 1 – locked

    void lock() {
        if (state.exchange(1, std::memory_order_acquire) == 0) {
            return;
        }

        do {
            futex_wait(&state, 1);
        } while (state.exchange(1, std::memory_order_acquire) != 0);
    }
}
```

```
void unlock() {  
    state.store(0, std::memory_order_release);  
    futex_wake(&state, 1);  
}  
};
```

Состояния:

0 – свободен
1 – занят

Идея хорошая:

если мьютекс свободен:
 захват через atomic exchange
 без syscall

если мьютекс занят:
 поток уходит в futex_wait
 не жжёт CPU

Но есть две проблемы:

1. unlock всегда делает futex_wake, даже если никто не ждёт
2. между store(0) и futex_wake возможен use-after-free

2. Проблема 1: лишние сисколлы

В двухсостояниеовой версии:

```
void unlock() {  
    state.store(0);  
    futex_wake(&state, 1);  
}
```

`futex_wake` вызывается всегда.

Но часто мьютекс используется без конкуренции:

```
thread A:  
    lock  
    critical section
```

```
unlock
```

никто другой не пытался захватить этот mutex

В таком случае:

```
ждущих потоков нет  
будить некого  
futex_wake — лишний syscall
```

А syscall дорогой:

```
переход user → kernel  
работа ядра  
возврат kernel → user
```

Хотелось бы:

```
если ждущих нет:  
    unlock просто освобождает mutex  
    без futex_wake  
  
если ждущие есть:  
    unlock освобождает mutex  
    делает futex_wake
```

Для этого мьютекс должен знать не только:

```
занят / свободен
```

но и:

```
есть ли ожидающие
```

3. Проблема 2: use-after-free между `store(0)` и `futex_wake`

Самая тонкая проблема:

```
void unlock() {  
    state.store(0);  
}
```

```
futex_wake(&state, 1);  
}
```

После:

```
state.store(0);
```

мьютекс уже свободен.

Другой поток может:

```
увидеть, что мьютекс свободен  
захватить его  
завершить работу с объектом  
уничтожить объект
```

А первый поток ещё не закончил `unlock`, потому что ему осталось выполнить:

```
futex_wake(&state, 1);
```

Но `state` уже может быть частью уничтоженного объекта.

Получается обращение к освобождённой памяти.

Это называется:

```
use-after-free
```

4. Пример `OneShot`

Пример из конспекта: есть объект, который должен выполнить функцию ровно один раз.

```
class OneShot {  
    std::mutex mtx;  
    bool called = false;  
  
public:  
    void call() {  
        std::lock_guard lk(mtx);  
  
        if (!called) {  
            called = true;  
            func();  
        }  
    }  
};
```

```
    }  
}  
};
```

Представим, что `std::mutex` внутри реализован нашей двухсостояниеовой схемой.

Сценарий:

```
thread 1:  
  вызывает os->call()  
  захватывает mtx  
  выполняет func()  
  выходит из lock_guard  
  начинается unlock()
```

Внутри `unlock`:

```
state.store(0);
```

Теперь мьютекс считается свободным.

Планировщик переключается.

```
thread 2:  
  видит, что call уже завершился  
  уничтожает объект OneShot  
  вместе с ним уничтожается поле mtx
```

Потом планировщик возвращается к thread 1.

```
thread 1:  
  продолжает unlock()  
  вызывает futex_wake(&state, 1)
```

Но `state` уже находился внутри уничтоженного `mtx`.

Итог:

```
  futex_wake обращается к адресу уничтоженного объекта  
  use-after-free  
  undefined behavior
```

Важная фраза:

Проблема не в том, что `futex_wake` обязательно упадёт. Проблема в том, что на уровне C++ мы обращаемся к памяти объекта после того, как он мог быть уничтожен.

5. Почему нельзя просто поменять порядок

Может показаться, что надо сделать так:

```
void unlock() {  
    futex_wake(&state, 1);  
    state.store(0);  
}
```

Но это ломает корректность.

Сценарий lost wakeup:

```
state = 1  
mutex занят
```

Поток 1 делает `unlock`:

```
thread 1:  
    futex_wake(&state, 1)
```

Но в этот момент ещё никто не ждёт.

```
wake ушёл в пустоту
```

Потом поток 1 делает:

```
state.store(0);
```

А другой поток мог как раз готовиться уснуть:

```
thread 2:  
    увидел, что state было 1  
    собирался вызвать futex_wait(&state, 1)
```

Если порядок неудачный, он может уснуть после того, как единственный wake уже произошёл.

```
thread 2:
    уходит в futex_wait
    будить больше некому
```

Получается lost wakeup.

Значит:

```
store должен быть до wake,
чтобы ожидающие увидели изменение состояния
```

Но тогда возвращается проблема use-after-free.

Именно поэтому нужен другой дизайн.

6. Попытка: отдельный счётчик ожидающих

Можно попробовать добавить отдельный счётчик:

```
std::atomic<int> state{0};
std::atomic<int> waiting{0};
```

Идея:

```
если waiting == 0:
    никого не будим

если waiting > 0:
    делаем futex_wake
```

Наивная реализация:

```
void lock() {
    if (state.exchange(1) == 0) {
        return;
    }

    waiting.fetch_add(1);

    while (state.exchange(1) == 1) {
        futex_wait(&state, 1);
    }

    waiting.fetch_sub(1);
}
```

```

}

void unlock() {
    state.store(0);

    if (waiting.load() > 0) {
        futex_wake(&state, 1);
    }
}

```

На первый взгляд хорошо:

```

ждущих нет → wake не зовём
ждущие есть → wake зовём

```

Но это ломается.

7. Почему отдельный `waiting` ломается

Проблема в том, что `state` и `waiting` — разные атомарные переменные.

Между ними нет единой атомарной операции.

Сценарий:

```

state = 1
waiting = 0

```

Поток 2 хочет захватить lock, но ещё не успел увеличить `waiting`.

Поток 1 делает `unlock`:

```

thread 1:
    state.store(0)
    waiting.load() == 0
    futex_wake не вызывает

```

Теперь поток 2:

```

thread 2:
    waiting.fetch_add(1)
    видит занятость / уходит в futex_wait

```

Но wake уже не будет.

Возможен lost wakeup.

Главный вывод:

Нельзя хранить состояние мьютекса и факт наличия ожидающих в двух независимых атомарных переменных. Их нужно объединить в одно атомарное слово и менять согласованно.

8. Идея трёх состояний

Вместо двух состояний:

0 — свободен
1 — занят

используем три:

0 — свободен
1 — занят, ждущих нет
2 — занят, есть или могли быть ждущие

Смысл:

0

mutex свободен
можно захватить без syscall

1

mutex занят
но никто ещё не заявил, что будет спать

Если `unlock` видит, что было состояние 1, значит:

ждущих нет
можно просто сделать 1 → 0
`futex_wake` не нужен

2

mutex занят
есть или могли быть ждущие

Если `unlock` видит, что было состояние 2, значит:

надо сделать `state = 0`
надо вызвать `futex_wake`

Важная мысль:

Состояние 2 — это консервативная пометка: “возможно, есть ожидающие, поэтому при `unlock` надо будить”.

9. Состояния как один атомик

```
std::atomic<int> state{0};
```

В одном слове хранится вся информация:

0 — free
1 — locked, no waiters
2 — locked, maybe waiters

Почему это лучше?

`state` и информация о `waiters` меняются одной атомарной переменной
нет гонки между отдельным `state` и отдельным `waiting`
`unlock` может понять, нужно ли `wake`

10. Реализация мьютекса на трёх состояниях

Код из конспекта:

```
struct Mutex3 {  
    std::atomic<int> state{0};  
  
    void lock() {  
        int c = 0;
```

```

// Fast path: 0 -> 1 без syscall
if (state.compare_exchange_strong(
    c,
    1,
    std::memory_order_acquire
)) {
    return;
}

// Slow path
do {
    // Если c == 2, уже известно, что есть/могут быть ждущие.
    // Если c != 2, переводим состояние в 2.
    if (c == 2 || state.exchange(2, std::memory_order_acquire) != 0)
{
        futex_wait(&state, 2);
    }

    // После пробуждения пробуем захватить:
    // хотим перевести 0 -> 2.
    c = 0;

} while (!state.compare_exchange_strong(
    c,
    2,
    std::memory_order_acquire
));
}

void unlock() {
    // Если было 1, fetch_sub делает 1 -> 0.
    // Если было 2, fetch_sub делает 2 -> 1, но нам надо освободить и
разбудить.
    if (state.fetch_sub(1, std::memory_order_release) != 1) {
        state.store(0, std::memory_order_release);
        futex_wake(&state, 1);
    }
}
};

```

Это не самый простой код для первого чтения, поэтому разберём его отдельно.

11. Быстрый путь `lock`

```

int c = 0;

if (state.compare_exchange_strong(c, 1)) {

```

```
    return;  
}
```

Мы пытаемся сделать:

```
0 → 1
```

То есть:

```
если mutex свободен:  
    сделать его занятым без waiters  
    войти в critical section
```

Если `state == 0`, CAS успешен:

```
state становится 1  
lock захвачен  
syscall нет
```

Это обычный uncontended path.

12. Если быстрый путь не сработал

Если CAS не сработал, значит `state` был не `0`.

То есть:

```
state == 1  
или  
state == 2
```

В переменную `c` при неудачном CAS записалось фактическое значение `state`.

Например:

```
c = 1
```

или:

```
c = 2
```

Дальше поток идёт в slow path.

13. Зачем переводить состояние в 2

Если поток обнаружил, что мьютекс занят, он должен сообщить владельцу:

когда будешь делать unlock, меня нужно разбудить

Для этого он переводит состояние в 2.

```
state.exchange(2)
```

Смысл:

мьютекс занят
теперь он помечен как contended
при unlock надо делать wake

Состояние 2 говорит владельцу:

при освобождении не просто делай $1 \rightarrow 0$,
а сделай wake

14. Почему `futex_wait(&state, 2)`

В slow path поток ждёт так:

```
futex_wait(&state, 2);
```

То есть:

спи, пока `state == 2`

Почему не 1?

Потому что ожидающие потоки сами переводят состояние в 2.

2 означает:

мьютекс занят и есть/могут быть ожидающие

Если `unlock` выполнится, он сделает:

```
state.store(0);  
futex_wake(&state, 1);
```

И ожидающий поток проснётся.

Если к моменту `futex_wait` состояние уже не `2`, `futex_wait` сразу вернётся, и поток снова попытается захватить lock.

15. Почему после пробуждения CAS делает `0 → 2`, а не `0 → 1`

В конце slow path:

```
c = 0;  
  
while (!state.compare_exchange_strong(c, 2)) {  
    ...  
}
```

Когда поток проснулся и увидел свободный mutex, он захватывает его переводом:

```
0 → 2
```

Почему не `0 → 1`?

Потому что раз мы попали в slow path, уже была конкуренция. Возможно, остались другие ожидающие потоки.

Если захватить как `1`, то при последующем `unlock` мы можем не сделать `wake`, хотя кто-то ещё ждёт.

Состояние `2` консервативно сохраняет информацию:

```
есть или могли быть другие waiters
```

Поэтому поток, пришедший из slow path, становится владельцем, но оставляет состояние:

```
locked, maybe waiters
```

16. `unlock`: быстрый случай

```
if (state.fetch_sub(1) != 1) {  
    ...  
}
```

Если `state` было `1`:

```
state.fetch_sub(1) вернёт 1  
state станет 0
```

Это значит:

```
мьютекс был занят  
ждущих нет  
мы освободили его  
wake не нужен
```

И функция `unlock` сразу заканчивается.

Это решает проблему лишних сисколлов:

```
uncontended unlock:  
    fetch_sub 1 → 0  
    no futex_wake
```

И важно для `OneShot`:

```
после fetch_sub дополнительных обращений к памяти мьютекса нет
```

То есть если объект уничтожат сразу после освобождения, текущий поток уже не будет делать `futex_wake(&state)`.

17. `unlock`: случай с ожидающими

Если `state` было `2`:

```
state.fetch_sub(1)
```

вернёт `2`, а `state` временно станет `1`.

Условие:

```
state.fetch_sub(1) != 1
```

истинно.

Значит, были или могли быть ожидающие.

Тогда делаем:

```
state.store(0);  
futex_wake(&state, 1);
```

Смысл:

освободить mutex
разбудить одного ожидающего

Почему можно будить одного?

mutex один
после освобождения войти сможет только один поток

18. Что именно решает трёхсостояниеовая схема

1. Уменьшение числа сисколлов

В двухсостояниеовой версии:

```
unlock всегда делает futex_wake
```

В трёхсостояниеовой:

```
state == 1:  
    ждущих нет  
    unlock не делает futex_wake  
  
state == 2:  
    есть/могут быть ждущие  
    unlock делает futex_wake
```

Это экономит syscall на неконкурентном пути.

2. Проблема OneShot

В типичном неконкурентном случае OneShot :

```
поток один раз зашёл в call  
выполнил func  
вышел из lock_guard
```

Состояние было:

```
state == 1
```

unlock делает:

```
fetch_sub(1)
```

и больше не обращается к памяти мьютекса.

Значит:

```
после освобождения мьютекса объект может быть уничтожен  
поток unlock уже не полезет в state через futex_wake
```

Это устраняет "проезд по памяти" в важном случае.

3. Согласованная информация о waiters

Вместо отдельного:

```
state  
waiting
```

мы используем одно атомарное слово:

```
state = 0 / 1 / 2
```

Поэтому нет гонки:

```
один поток проверил waiting == 0
другой только потом увеличил waiting
и уснул навсегда
```

Состояние `2` выставляется как часть единой логики захвата.

19. Почему состояние `2` означает “есть или могли быть ждущие”

Важно не понимать `2` слишком буквально.

`2` не гарантирует, что прямо сейчас кто-то действительно спит в `futex`.

Возможны случаи:

```
кто-то собирался ждать, но ещё не уснул
кто-то уже проснулся
кто-то был разбужен
ждущих уже нет, но state всё ещё 2
```

Поэтому лучше говорить:

```
2 – contended state
2 – locked, maybe waiters
2 – нужно сделать wake при unlock
```

Это консервативная пометка.

Она может привести к лишнему `futex_wake`, но не к потерянному пробуждению.

20. Почему это всё ещё не “идеальный код для продакшена”

Этот код учебный.

В реальных `pthread_mutex` / `glibc` могут быть дополнительные детали:

```
адаптивное spinning перед futex_wait
разные типы mutex
robust mutex
priority inheritance
разные memory order
```

оптимизации fast path
обработка сигналов и ошибок futex

Но идея та же:

одно атомарное слово
быстрый uncontended path без syscall
contended state
futex_wait только при конкуренции
futex_wake только если есть/могли быть waiters

21. Сравнение двухсостояниевого и трёхсостояниевого мьютекса

Свойство	2 состояния	3 состояния
Состояния	0 free, 1 locked	0 free, 1 locked no waiters, 2 locked maybe waiters
Fast lock	exchange	CAS 0→1
Unlock без waiters	store(0) + wake	fetch_sub 1→0, без wake
Лишние syscalls	больше	меньше
Информация о waiters	нет	есть в state
Проблема OneShot	есть	снята в типичном uncontended case
Сложность	проще	сложнее

22. Короткий ответ на экзамене

Можно отвечать так:

В двухсостояниевоm futex-мьютексе есть только состояния 0 — свободен и 1 — занят. В unlock мы делаем state.store(0) и затем futex_wake. Это показывает идею, но имеет две проблемы. Во-первых, futex_wake вызывается всегда, даже если никто не ждёт, то есть мы делаем лишний syscall. Во-вторых, после store(0) мьютекс уже свободен, и другой поток может захватить его, завершить работу с объектом и уничтожить объект, пока первый поток ещё не вызвал futex_wake. Тогда futex_wake обращается к памяти уже уничтоженного мьютекса. Это проблема OneShot.

Переставить порядок на `wake` перед `store` нельзя, потому что тогда можно получить lost wakeup: `wake` уйдёт до того, как поток реально уснул, потом состояние станет свободным, а ожидающий поток заснёт и его уже никому будет будить.

Попытка добавить отдельный счётчик ожидающих тоже ломается, потому что `state` и `waiting` — две разные атомарные переменные. Можно проверить `waiting == 0`, не сделать `wake`, а другой поток сразу после этого увеличит `waiting` и уйдёт спать.

Поэтому состояние и информацию о ждущих объединяют в одно атомарное слово с тремя состояниями: `0` — свободно, `1` — занято без ожидающих, `2` — занято, есть или могут быть ожидающие. Быстрый `lock` делает CAS `0→1`. Если `lock` занят, поток переводит состояние в `2` и ждёт через `futex_wait(&state, 2)`. В `unlock` делаем `fetch_sub(1)`: если было `1`, стало `0`, ждущих нет, syscall не нужен. Если было `2`, значит были или могли быть ожидающие, тогда делаем `store(0)` и `futex_wake`.

Так трёхсостояниева схема уменьшает число сисколлов на неконкурентном пути и убирает обращение к памяти мьютекса после освобождения в типичном случае `OneShot`.

23. Мини-код для ответа на доске

```
struct Mutex3 {
    std::atomic<int> state{0};
    // 0 – free
    // 1 – locked, no waiters
    // 2 – locked, maybe waiters

    void lock() {
        int c = 0;

        // fast path: free -> locked without waiters
        if (state.compare_exchange_strong(
            c,
            1,
            std::memory_order_acquire
        )) {
            return;
        }

        // slow path
        do {
            if (c == 2 ||
                state.exchange(2, std::memory_order_acquire) != 0) {
                futex_wait(&state, 2);
            }
        } while (true);
    }
};
```

```

        c = 0;

    } while (!state.compare_exchange_strong(
        c,
        2,
        std::memory_order_acquire
    ));
}

void unlock() {
    if (state.fetch_sub(1, std::memory_order_release) != 1) {
        state.store(0, std::memory_order_release);
        futex_wake(&state, 1);
    }
}
};

```

Что объяснить:

0 – свободен

1 – занят, ждущих нет

2 – занят, есть/могут быть ждущие

CAS 0→1 – быстрый lock без syscall

переход в 2 – поток сообщает владельцу, что есть конкуренция

futex_wait(&state, 2) – спим, пока состояние contended

unlock из 1 – fetch_sub делает 1→0, wake не нужен

unlock из 2 – освобождаем через store(0) и будим одного

24. Мини-проверка

Сделаю коротко — 12 вопросов.

1. Какие проблемы есть у двухсостояниевого futex-мьютекса?
2. Почему `futex_wake` в `unlock` может быть лишним syscall?
3. В чём суть проблемы `OneShot`?
4. Почему нельзя просто поменять порядок на `futex_wake`, потом `store(0)`?
5. Почему отдельный счётчик `waiting` не решает проблему?
6. Какие три состояния используются в `Mutex3`?
7. Что означает состояние `1`?
8. Что означает состояние `2`?
9. Что делает быстрый путь `lock`?
10. Почему поток в slow path переводит состояние в `2`?
11. Что происходит в `unlock`, если старое состояние было `1`?
12. Что происходит в `unlock`, если старое состояние было `2`?